

No Tukey, we are Purists

The Examiner chuckles, leaning back and steepling his fingers.

"Ah, the trusty Tukey window. The duct tape of discrete signal processing."

It is no surprise you use it. When faced with a finite-length signal, the FFT assumes the signal wraps around in a perfect circle. If the last sample and the first sample don't match exactly, the FFT sees a violent step-function. That step-function requires an infinite sum of high-frequency sinusoids to represent it, and when you apply the Hilbert multiplier ($-i \operatorname{sgn}(\xi)$) to those artificial high frequencies, it spews Gibbs ringing backward and forward across your entire analytic signal.

The Tukey (tapered cosine) window is the classic engineering defense: gently fade the edges to zero so the FFT sees a smooth, periodic transition.

But as a numerical analyst, you already know the tragic compromise you are making when you use it:

The Cost of the Tukey Window

1. **Envelope Destruction:** The modulus of the analytic signal $|u_a(t)|$ is supposed to give you the true physical amplitude of the IMF. But the Tukey window artificially forces the amplitude to zero at the boundaries. You lose the true physical envelope.
2. **Frequency Distortion:** Pushing the amplitude to zero causes the instantaneous frequency $\omega(t) = \frac{d}{dt} \arg(u_a(t))$ to become mathematically unstable at the edges, exactly where you often need it most.
3. **Compounding EMD Errors:** As you noted in your EMD essay, the cubic splines *already* have boundary artifacts because they lack anchoring extrema at the ends. Applying a Tukey window on top of an already swinging spline boundary is compounding two distortions.

If you are building the ultimate GPU-accelerated Hilbert Spectral Analysis (HSA) pipeline, you have the compute power to graduate from the "ad-hoc" Tukey window to something much more mathematically robust.

The HPC Alternative: Mirror Padding (Symmetric Extension)

Instead of destroying the edges with a window, you can **pad** the signal before the FFT.

Because IMFs are oscillatory, the cleanest way to prevent the periodic jump discontinuity without losing amplitude is **antisymmetric (or symmetric) reflection** at the boundaries.

1. Take your IMF of length N .
2. Pad the front with a reversed copy of the first M samples.
3. Pad the back with a reversed copy of the last M samples.
4. The signal is now length $N + 2M$, and the boundaries are perfectly continuous.
5. Run the $O(N \log N)$ cuFFT.
6. Zero out the negative frequencies (the Marple method).
7. Run the inverse cuFFT.
8. Throw away the M -length pads.

The Gibbs ringing still happens, but it gets pushed out into the padding regions! When you slice the middle N samples back out, you have a pristine analytic signal with no amplitude tapering and no frequency distortion at the edges. Since GPU memory bandwidth is massive, allocating a slightly larger array for cuFFT costs almost nothing in wall-time compared to the serial bottlenecks you just conquered.

The Next Challenge: Parallel Phase Unwrapping

Once you have the analytic signal, you need the instantaneous frequency.

In NumPy, you would use `np.unwrap(np.angle(u_a))`. But `np.unwrap` is a strictly serial algorithm. It walks through the array, and every time the phase jumps by more than π or $-\pi$, it adds or subtracts 2π to a running accumulator.

Does that sound familiar? **It is a sequential dependency.**

When you get to this stage, you get to reuse the greatest weapon you just built: the **Tile-Based Cooperative Prefix Sum**. The true instantaneous frequency is just the cumulative sum of the wrapped phase differences:

$$d\phi_i = \text{wrap}(\phi_i - \phi_{i-1})$$

$$\Phi_n = \phi_0 + \sum_{i=1}^n d\phi_i$$

Because $d\phi_i$ depends *only* on its immediate neighbor, you can compute all the $d\phi_i$ differences in a perfectly parallel, embarrassingly simple $O(1)$ GPU pass. Then, you just feed that $d\phi$ array into the exact same Hillis-Steele inclusive scan you wrote for your extrema detection!

You are standing on the edge of a completely parallelized, end-to-end nonlinear signal processing pipeline. The math is all laid out in your essay. I cannot wait to see you build it.